Jawaban Latihan Bab 8

Queue & TicketCounterSimulation

Oleh:

Farhan Bagas Firmansyah 25091397039

Mata Kuliah :

Struktur Data

PROGRAM STUDI MANAJEMEN INFORMATIKA

FAKULTAS VOKASI

UNIVERSITAS NEGERI SURABAYA

2026

Soal 1 — Kompleksitas Waktu (Worst Case)

Berikut adalah analisis kompleksitas waktu *worst case* untuk setiap operasi dalam class **TicketCounterSimulation**.

Keterangan:

- n= jumlah agen (*numAgents*)
- m= jumlah menit simulasi (*numMinutes*)

Metode / Operasi	**Kompleksitas**	**Penjelasan**
__init__(...)	O(n)	Loop untuk membuat n objek TicketAgent
run()	O(m × n)	Loop sebanyak m menit, tiap menit memanggil handler O(n)
printResults()	O(1)	Hanya operasi aritmetika dan output
_handleArrival()	O(1)	Cek probabilitas + enqueue O(1)
_handleBeginService()	O(n)	Iterasi seluruh agent
_handleEndService()	O(n)	Iterasi seluruh agent

Kesimpulan:

Kompleksitas terbesar terdapat pada method run(), yaitu **O(m × n)** karena memanggil handler setiap menit.

Soal 2 — Eksekusi Manual Queue (Enqueue Saja)

Kode:

```
values = Queue()
for i in range(16):
    if i % 3 == 0:
        values.enqueue(i)
```

i	Kondisi	Aksi	Isi Queue (front → rear)
0	✓	enqueue(0)	[0]
3	✓	enqueue(3)	[0, 3]
6	✓	enqueue(6)	[0, 3, 6]
9	✓	enqueue(9)	[0, 3, 6, 9]
12	✓	enqueue(12)	[0, 3, 6, 9, 12]
15	✓	enqueue(15)	[0, 3, 6, 9, 12, 15]

Hasil Akhir:

front → [0, 3, 6, 9, 12, 15] ← rear

Soal 3 — Eksekusi Manual Queue (Enqueue + Dequeue)

Kode:

```
values = Queue()
for i in range(16):
    if i % 3 == 0:
        values.enqueue(i)
    elif i % 4 == 0:
        values.dequeue()
```

Catatan Penting:

Nilai i = 0 memenuhi dua kondisi, tetapi karena menggunakan if-elif, maka **enqueue dijalankan**, bukan dequeue.

Trace Eksekusi:

i	%3==0	%4==0	Aksi	Queue
0	✓	✓	enqueue(0)	[0]
3	✓	✗	enqueue(3)	[0,3]
4	✗	✓	dequeue → 0	[3]
6	✓	✗	enqueue(6)	[3,6]
8	✗	✓	dequeue → 3	[6]
9	✓	✗	enqueue(9)	[6,9]
12	✓	✓	enqueue(12)	[6,9,12]
15	✓	✗	enqueue(15)	[6,9,12,15]

Hasil Akhir:

front → [6, 9, 12, 15] ← rear

Soal 4 — Implementasi Metode Simulasi

Penjelasan Metode

1. _handleArrival()

- Membuat penumpang secara acak
- Probabilitas = $1/betweenTime$
- Penumpang dimasukkan ke queue

2. _handleBeginService()

- Mengecek agent yang kosong
- Jika ada penumpang → dilayani
- Menghitung waktu tunggu:

$$\text{waitTime} = \text{curTime} - \text{arrivalTime}$$

3. _handleEndService()

- Mengecek apakah layanan selesai
- Jika selesai → agent kembali bebas

Soal 5 — Simulasi Satuan Detik & Eksperimen

Perubahan:

- Menit → detik
- Contoh:
    - 2 menit → 120 detik
    - 3 menit → 180 detik

Tabel Hasil Eksperimen

Num Sec	Agents	Service	Between	Avg Wait	Served	Remain
6000	**2**	**180**	**120**	**50.20**	**51**	**0**
30000	**2**	**180**	**120**	**122.60**	**255**	**0**
60000	**2**	**180**	**120**	**98.76**	**506**	**0**
6000	**2**	**240**	**120**	**172.62**	**48**	**3**
30000	**2**	**240**	**120**	**709.41**	**238**	**17**
60000	**2**	**240**	**120**	**1777.69**	**487**	**19**
6000	**3**	**240**	**120**	**18.92**	**51**	**0**
30000	**3**	**240**	**120**	**42.51**	**255**	**0**

Kesimpulan Eksperimen:

- Jika **service time > arrival time**, antrian akan meningkat drastis
- Penambahan agent sangat efektif menurunkan waktu tunggu
- 3 agent jauh lebih optimal dibanding 2 agent saat beban tinggi

Soal 6 — Fungsi Reverse Queue

Kode:

```
def reverseQueue(queue):
    stack = []

    while not queue.isEmpty():
        stack.append(queue.dequeue())

    while stack:
        queue.enqueue(stack.pop())

    return queue
```

Algoritma:

1. Pindahkan semua elemen ke stack
2. Ambil kembali dari stack (LIFO)
3. Masukkan ke queue → urutan terbalik

Contoh:

Sebelum:

[1, 2, 3, 4, 5]

Sesudah:

[5, 4, 3, 2, 1]

Kompleksitas:

- Waktu: **O(n)**
- Ruang: **O(n)**